

LAB EXPERIMENTS

ECA1408-EMBEDDED SYSTEMS

Name: Aadhithya V

Reg.No.:192511256

Exp-24:

Code:

```
#include <systemc.h>
```

```
SC_MODULE(Design) {  
    sc_in<bool> clk;
```

```
    SC_CTOR(Design) {  
        SC_THREAD(task1);  
        SC_THREAD(task2);  
    }
```

```
    void task1() {  
        for (int i = 1; i <= 3; i++) {  
            wait(clk.posedge_event());  
            cout << sc_time_stamp()  
                << " Task 1: " << i << endl;  
        }  
    }
```

```
    void task2() {
```

```
int sc_main(int argc, char* argv[]) {
    sc_clock clk("clk", 10, SC_NS);

    Design d("d");

    d.clk(clk);

    sc_start(40, SC_NS);

    return 0;
}
```

The screenshot shows the EDA Playground web interface. The top navigation bar includes links for 'New', 'Run', 'Save', 'Copy', 'KnowHow', 'Building Embedded Products with Zephyr', 'REGISTER NOW', 'Resources', 'Community', 'Help', 'Playgrounds', and 'Profile'. The left sidebar contains sections for 'Languages & Libraries' (with 'C++' selected), 'Tools & Simulators' (with 'Zephyr' selected), and 'Examples'. The main area is divided into two panes. The left pane shows a C++ code file named 'main.cpp' with the following content:

```

1 // main.cpp
2 #include <zephyr.h>
3 #include <stdio.h>
4 #include <sys/time.h>
5
6 #include <zephyr.h>
7
8 #include <zephyr.h>
9
10 #include <zephyr.h>
11
12 #include <zephyr.h>
13
14 #include <zephyr.h>
15
16 #include <zephyr.h>
17
18 #include <zephyr.h>
19
20 #include <zephyr.h>
21
22 #include <zephyr.h>
23
24 #include <zephyr.h>
25
26 #include <zephyr.h>
27
28 #include <zephyr.h>
29
30 #include <zephyr.h>
31
32 #include <zephyr.h>
33
34 #include <zephyr.h>
35
36 #include <zephyr.h>
37
38 #include <zephyr.h>
39
40 #include <zephyr.h>
41
42 #include <zephyr.h>
43
44 #include <zephyr.h>
45
46 #include <zephyr.h>
47
48 #include <zephyr.h>
49
50 #include <zephyr.h>
51
52 #include <zephyr.h>
53
54 #include <zephyr.h>
55
56 #include <zephyr.h>
57
58 #include <zephyr.h>
59
60 #include <zephyr.h>
61
62 #include <zephyr.h>
63
64 #include <zephyr.h>
65
66 #include <zephyr.h>
67
68 #include <zephyr.h>
69
70 #include <zephyr.h>
71
72 #include <zephyr.h>
73
74 #include <zephyr.h>
75
76 #include <zephyr.h>
77
78 #include <zephyr.h>
79
80 #include <zephyr.h>
81
82 #include <zephyr.h>
83
84 #include <zephyr.h>
85
86 #include <zephyr.h>
87
88 #include <zephyr.h>
89
90 #include <zephyr.h>
91
92 #include <zephyr.h>
93
94 #include <zephyr.h>
95
96 #include <zephyr.h>
97
98 #include <zephyr.h>
99
100 #include <zephyr.h>
101
102 #include <zephyr.h>
103
104 #include <zephyr.h>
105
106 #include <zephyr.h>
107
108 #include <zephyr.h>
109
110 #include <zephyr.h>
111
112 #include <zephyr.h>
113
114 #include <zephyr.h>
115
116 #include <zephyr.h>
117
118 #include <zephyr.h>
119
120 #include <zephyr.h>
121
122 #include <zephyr.h>
123
124 #include <zephyr.h>
125
126 #include <zephyr.h>
127
128 #include <zephyr.h>
129
130 #include <zephyr.h>
131
132 #include <zephyr.h>
133
134 #include <zephyr.h>
135
136 #include <zephyr.h>
137
138 #include <zephyr.h>
139
140 #include <zephyr.h>
141
142 #include <zephyr.h>
143
144 #include <zephyr.h>
145
146 #include <zephyr.h>
147
148 #include <zephyr.h>
149
150 #include <zephyr.h>
151
152 #include <zephyr.h>
153
154 #include <zephyr.h>
155
156 #include <zephyr.h>
157
158 #include <zephyr.h>
159
160 #include <zephyr.h>
161
162 #include <zephyr.h>
163
164 #include <zephyr.h>
165
166 #include <zephyr.h>
167
168 #include <zephyr.h>
169
170 #include <zephyr.h>
171
172 #include <zephyr.h>
173
174 #include <zephyr.h>
175
176 #include <zephyr.h>
177
178 #include <zephyr.h>
179
180 #include <zephyr.h>
181
182 #include <zephyr.h>
183
184 #include <zephyr.h>
185
186 #include <zephyr.h>
187
188 #include <zephyr.h>
189
190 #include <zephyr.h>
191
192 #include <zephyr.h>
193
194 #include <zephyr.h>
195
196 #include <zephyr.h>
197
198 #include <zephyr.h>
199
200 #include <zephyr.h>
201
202 #include <zephyr.h>
203
204 #include <zephyr.h>
205
206 #include <zephyr.h>
207
208 #include <zephyr.h>
209
210 #include <zephyr.h>
211
212 #include <zephyr.h>
213
214 #include <zephyr.h>
215
216 #include <zephyr.h>
217
218 #include <zephyr.h>
219
220 #include <zephyr.h>
221
222 #include <zephyr.h>
223
224 #include <zephyr.h>
225
226 #include <zephyr.h>
227
228 #include <zephyr.h>
229
230 #include <zephyr.h>
231
232 #include <zephyr.h>
233
234 #include <zephyr.h>
235
236 #include <zephyr.h>
237
238 #include <zephyr.h>
239
240 #include <zephyr.h>
241
242 #include <zephyr.h>
243
244 #include <zephyr.h>
245
246 #include <zephyr.h>
247
248 #include <zephyr.h>
249
250 #include <zephyr.h>
251
252 #include <zephyr.h>
253
254 #include <zephyr.h>
255
256 #include <zephyr.h>
257
258 #include <zephyr.h>
259
260 #include <zephyr.h>
261
262 #include <zephyr.h>
263
264 #include <zephyr.h>
265
266 #include <zephyr.h>
267
268 #include <zephyr.h>
269
270 #include <zephyr.h>
271
272 #include <zephyr.h>
273
274 #include <zephyr.h>
275
276 #include <zephyr.h>
277
278 #include <zephyr.h>
279
280 #include <zephyr.h>
281
282 #include <zephyr.h>
283
284 #include <zephyr.h>
285
286 #include <zephyr.h>
287
288 #include <zephyr.h>
289
290 #include <zephyr.h>
291
292 #include <zephyr.h>
293
294 #include <zephyr.h>
295
296 #include <zephyr.h>
297
298 #include <zephyr.h>
299
300 #include <zephyr.h>
301
302 #include <zephyr.h>
303
304 #include <zephyr.h>
305
306 #include <zephyr.h>
307
308 #include <zephyr.h>
309
310 #include <zephyr.h>
311
312 #include <zephyr.h>
313
314 #include <zephyr.h>
315
316 #include <zephyr.h>
317
318 #include <zephyr.h>
319
320 #include <zephyr.h>
321
322 #include <zephyr.h>
323
324 #include <zephyr.h>
325
326 #include <zephyr.h>
327
328 #include <zephyr.h>
329
330 #include <zephyr.h>
331
332 #include <zephyr.h>
333
334 #include <zephyr.h>
335
336 #include <zephyr.h>
337
338 #include <zephyr.h>
339
340 #include <zephyr.h>
341
342 #include <zephyr.h>
343
344 #include <zephyr.h>
345
346 #include <zephyr.h>
347
348 #include <zephyr.h>
349
350 #include <zephyr.h>
351
352 #include <zephyr.h>
353
354 #include <zephyr.h>
355
356 #include <zephyr.h>
357
358 #include <zephyr.h>
359
360 #include <zephyr.h>
361
362 #include <zephyr.h>
363
364 #include <zephyr.h>
365
366 #include <zephyr.h>
367
368 #include <zephyr.h>
369
370 #include <zephyr.h>
371
372 #include <zephyr.h>
373
374 #include <zephyr.h>
375
376 #include <zephyr.h>
377
378 #include <zephyr.h>
379
380 #include <zephyr.h>
381
382 #include <zephyr.h>
383
384 #include <zephyr.h>
385
386 #include <zephyr.h>
387
388 #include <zephyr.h>
389
390 #include <zephyr.h>
391
392 #include <zephyr.h>
393
394 #include <zephyr.h>
395
396 #include <zephyr.h>
397
398 #include <zephyr.h>
399
400 #include <zephyr.h>
401
402 #include <zephyr.h>
403
404 #include <zephyr.h>
405
406 #include <zephyr.h>
407
408 #include <zephyr.h>
409
410 #include <zephyr.h>
411
412 #include <zephyr.h>
413
414 #include <zephyr.h>
415
416 #include <zephyr.h>
417
418 #include <zephyr.h>
419
420 #include <zephyr.h>
421
422 #include <zephyr.h>
423
424 #include <zephyr.h>
425
426 #include <zephyr.h>
427
428 #include <zephyr.h>
429
430 #include <zephyr.h>
431
432 #include <zephyr.h>
433
434 #include <zephyr.h>
435
436 #include <zephyr.h>
437
438 #include <zephyr.h>
439
440 #include <zephyr.h>
441
442 #include <zephyr.h>
443
444
```

Exp-25:

Code:

```
#include <systemc.h>
```

```
struct Transaction {  
    int address;  
    int data;  
};
```

```
SC_MODULE(Receiver) {  
    void receive(Transaction t) {  
        cout << "Address: " << t.address << endl;  
        cout << "Data: " << t.data << endl;  
    }  
}
```

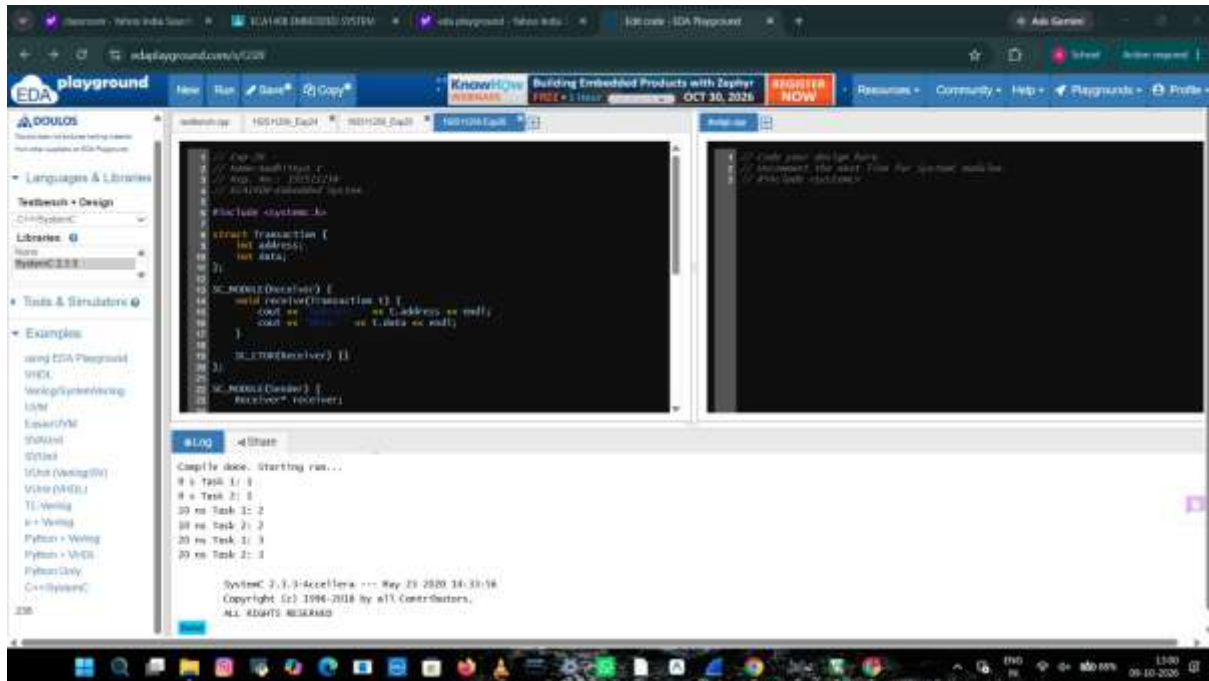
```
    SC_CTOR(Receiver) {}  
};
```

```
SC_MODULE(Sender) {  
    Receiver* receiver;  
  
    SC_CTOR(Sender) {  
        SC_THREAD(send);  
    }  
}
```

```
void send() {  
    Transaction t;  
    t.address = 100;  
    t.data = 250;  
  
    wait(5, SC_NS);  
    cout << "Sending transaction" << endl;  
    receiver->receive(t);  
}  
};
```

```
int sc_main(int argc, char* argv[]) {  
    Receiver r("receiver");  
    Sender s("sender");  
    s.receiver = &r;  
  
    sc_start(10, SC_NS);  
    return 0;  
}
```

Result:



Exp-26:

Code:

```
#include <systemc.h>
```

```
SC_MODULE(ProducerConsumer) {
```

```
    sc_fifo<int> fifo;
```

```
    SC_CTOR(ProducerConsumer) : fifo("fifo", 2) {
```

```
        SC_THREAD(producer);
```

```
        SC_THREAD(consumer);
```

```
    }
```

```
    void producer() {
```

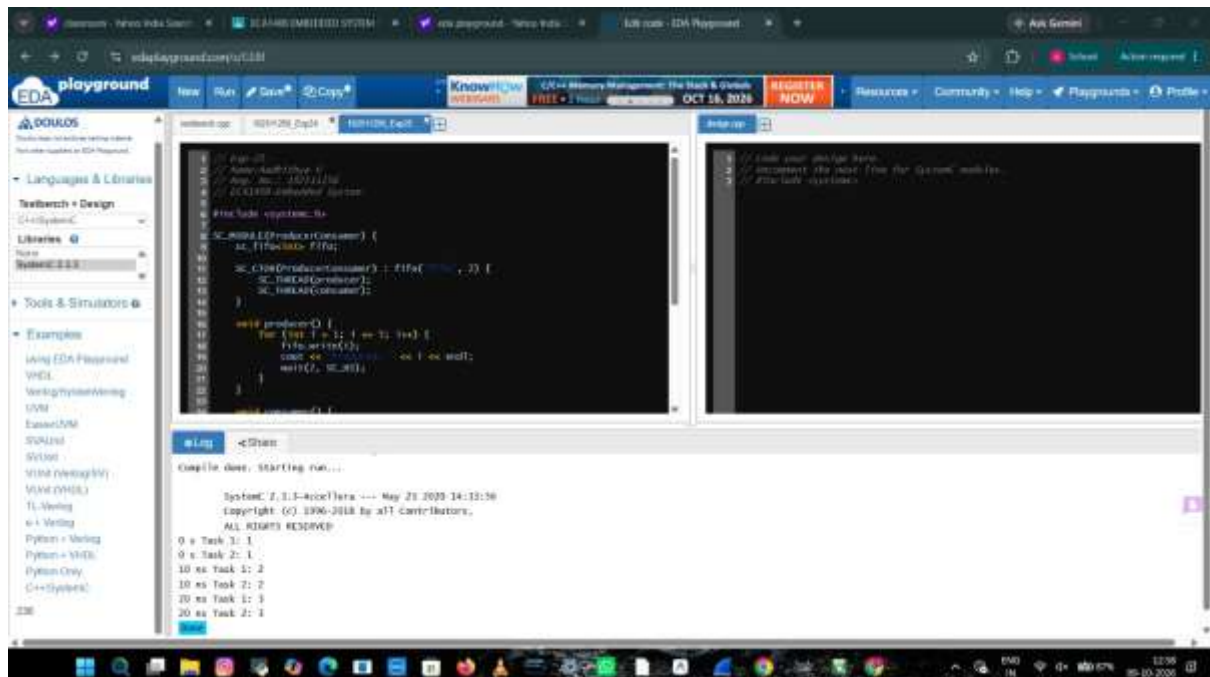
```
        for (int i = 1; i <= 5; i++) {
```

```
    fifo.write(i);  
    cout << "Produced: " << i << endl;  
    wait(2, SC_NS);  
}  
}
```

```
void consumer() {  
    for (int i = 1; i <= 5; i++) {  
        int data = fifo.read();  
        cout << "Consumed: " << data << endl;  
        wait(3, SC_NS);  
    }  
}  
};
```

```
int sc_main(int argc, char* argv[]) {  
    ProducerConsumer pc("pc");  
    sc_start(30, SC_NS);  
    return 0;  
}
```

Result:



Exp-27:

Code:

```
#include <systemc.h>
```

```
SC_MODULE(SemaphoreDemo) {
```

```
    sc_semaphore sem;
```

```
    SC_CTOR(SemaphoreDemo) : sem("sem", 1) {
```

```
        SC_THREAD(task1);
```

```
        SC_THREAD(task2);
```

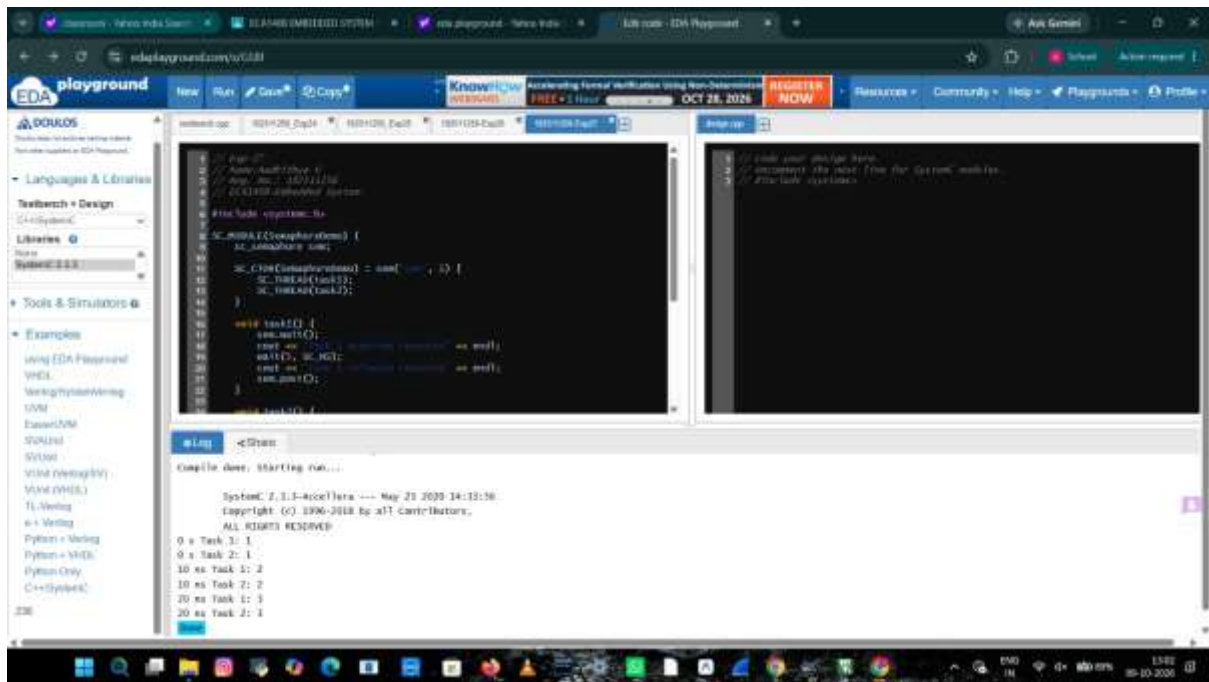
```
    }
```

```
void task1() {  
    sem.wait();  
    cout << "Task 1 acquired resource" << endl;  
    wait(5, SC_NS);  
    cout << "Task 1 released resource" << endl;  
    sem.post();  
}
```

```
void task2() {  
    wait(1, SC_NS);  
    sem.wait();  
    cout << "Task 2 acquired resource" << endl;  
    wait(2, SC_NS);  
    cout << "Task 2 released resource" << endl;  
    sem.post();  
}  
};
```

```
int sc_main(int argc, char* argv[]) {  
    SemaphoreDemo d("d");  
    sc_start(15, SC_NS);  
    return 0;  
}
```

Result:



Exp-28:

Code:

```
#include <systemc.h>
```

```
SC_MODULE(CountingDemo) {
```

```
    sc_semaphore sem;
```

```
    SC_CTOR(CountingDemo) : sem("sem", 2) {
```

```
        SC_THREAD(task1);
```

```
        SC_THREAD(task2);
```

```
        SC_THREAD(task3);
```

```
    }
```

```
    void task(int id) {
```

```
        sem.wait();
```


Exp-29:

Code:

```
#include <systemc.h>
```

```
SC_MODULE(InterruptDemo) {  
    bool high_pending, low_pending;
```

```
    SC_CTOR(InterruptDemo) {  
        high_pending = false;  
        low_pending = false;  
        SC_THREAD(run);  
    }
```

```
    void run() {  
        wait(5, SC_NS);  
  
        low_pending = true;  
        high_pending = true;
```

```
        cout << "Interrupts received" << endl;
```

```
        if (high_pending) {  
            cout << "High-priority interrupt serviced" << endl;  
            high_pending = false;  
        }
```

```

    if (low_pending) {
        cout << "Low-priority interrupt serviced" << endl;
        low_pending = false;
    }
}

};

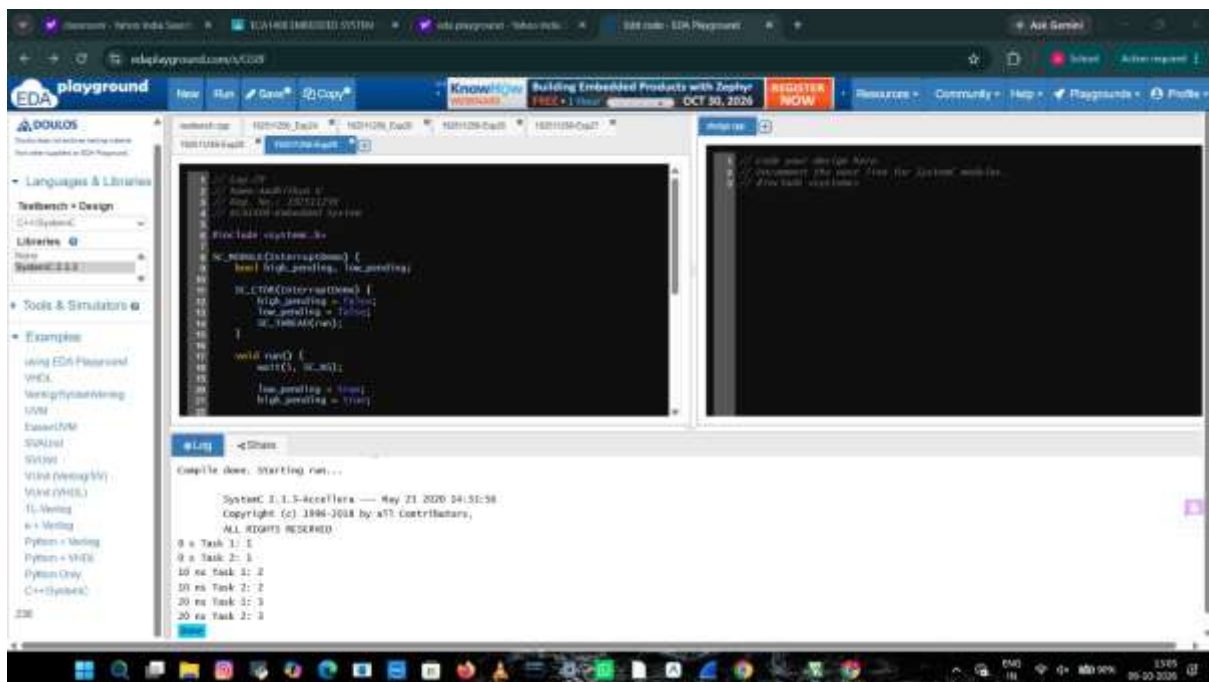
```

```

int sc_main(int argc, char* argv[]) {
    InterruptDemo d("d");
    sc_start(10, SC_NS);
    return 0;
}

```

Result:



Exp-30:

Code:

```
#include <systemc.h>
```

```
SC_MODULE(RTOSDemo) {  
    SC_CTOR(RTOSDemo) {  
        SC_THREAD(scheduler);  
    }  
}
```

```
void scheduler() {  
    cout << "RTOS Scheduler Started" << endl;  
  
    wait(1, SC_NS);  
    cout << "High Priority Task Executed" << endl;  
  
    wait(1, SC_NS);  
    cout << "Medium Priority Task Executed" << endl;  
  
    wait(1, SC_NS);  
    cout << "Low Priority Task Executed" << endl;  
  
    cout << "Scheduling Completed" << endl;  
}  
};
```

```
int sc_main(int argc, char* argv[]) {  
    RTOSDemo d("d");  
    sc_start(5, SC_NS);  
    return 0;  
}
```

Result:

